Unified Modeling Language (UML) Suite

Project: AutoTestHub (Web Automation & Testing Platform)

CLASS DIAGRAM • SEQUENCE DIAGRAM • ACTIVITY DIAGRAM

UML SPECIFICATION METADATA

System: AutoTestHub

Modeling Standard: OMG UML 2.5 Specification

Key Artifacts: Class Diagram, Sequence Diagram, Activity Diagram

Milestone: Mid-Semester Review

PROJECT DEVELOPMENT TEAM

Shristi Neekhra (Integration Lead)

Shifa Siddiqui (Backend Architect)

Shruti Gupta (Selenium Engine Architect)

Shriya Agarwal (Frontend Specialist)

Supervisor: [Supervisor Name]

1. System Class Diagram

The system class diagram details the object-oriented structure across both the Node.js API services and the Python Selenium automation worker modules, illustrating class attributes, method signatures, and relationships.

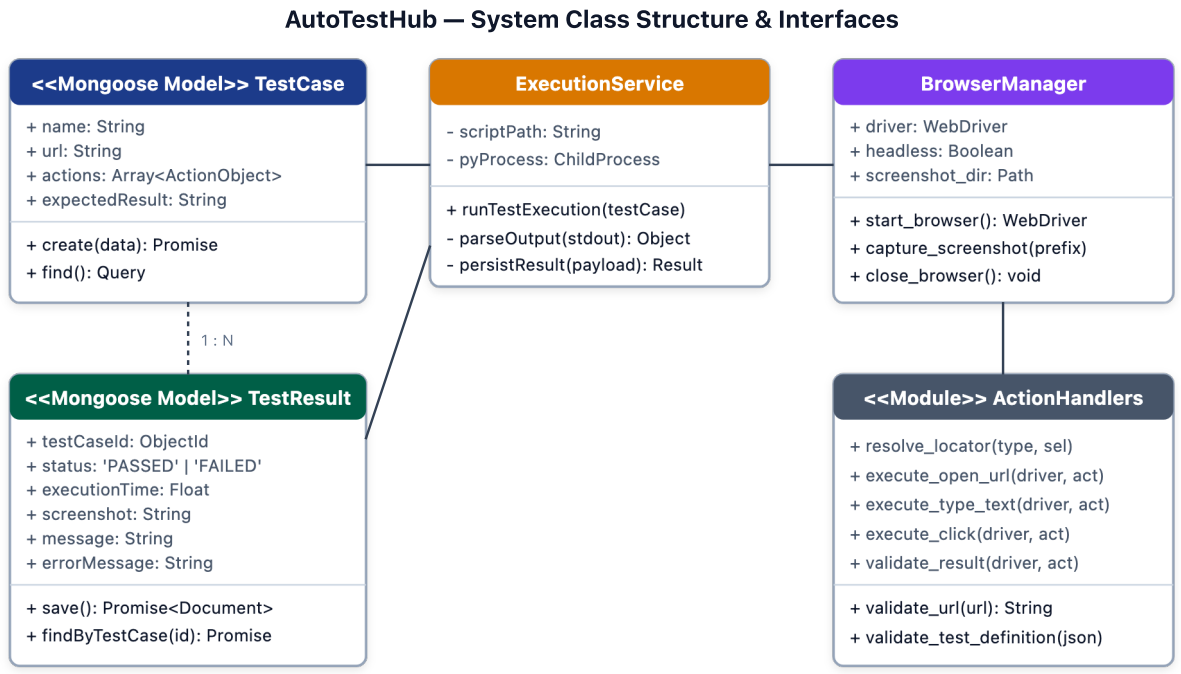


Figure 1: AutoTestHub Class Diagram illustrating Node.js Mongoose models, Execution Service, and Python Worker classes.

1.1 Class Architecture & Interfaces

- `TestCase` and `TestResult`: Mongoose model classes managing database schema definitions, validation, and queries.
- `ExecutionService`: Coordinates subprocess lifecycle, streams JSON payloads across standard streams, and parses execution output.
- `BrowserManager`: Manages the Chrome WebDriver session, encapsulates window maximize / headless options, and handles screenshot capture.
- `ActionHandlers`: Static module housing locator resolution and explicit-wait action executors for navigation, text input, clicks, and title assertions.

2. Execution Sequence Diagram

The execution sequence diagram models the temporal, chronological message exchange among system components during an end-to-end test execution triggered via the REST API.

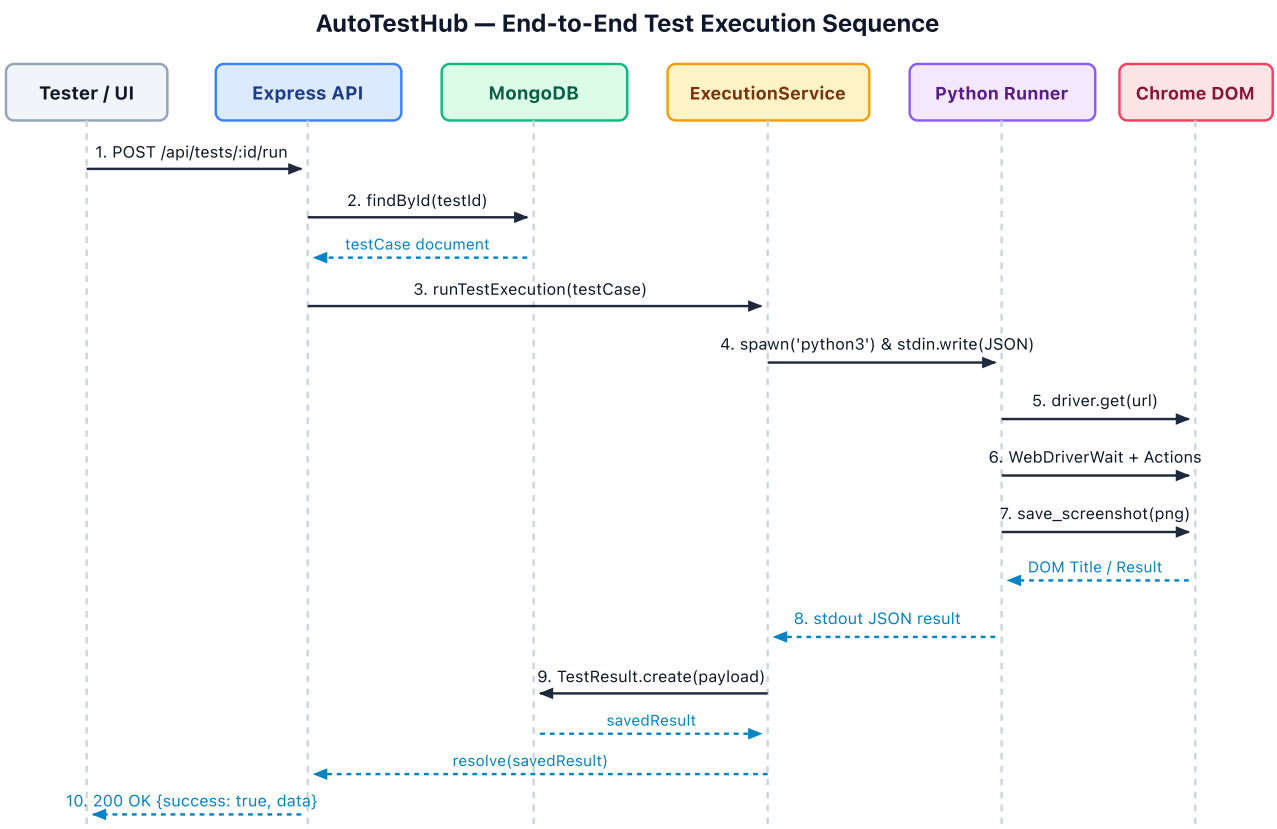


Figure 2: Sequence Diagram illustrating message flow from User trigger to Chrome DOM and MongoDB persistence.

2.1 Execution Sequence Walkthrough

1. **Trigger:** The client dispatches an HTTP `POST /api/tests/:id/run` request to the Express API.
2. **Query:** The test controller queries MongoDB via `TestCase.findById(testId)` to retrieve the target test definition.
3. **Dispatch:** The controller delegates the test document to `runTestExecution(testCase)`.
4. **IPC Launch:** The execution service spawns `python3 runner.py`, writing the JSON payload directly into its `stdin`.
5. **Automation:** Python parses the JSON, initializes Chrome WebDriver via Selenium Manager, navigates to the URL, and executes actions sequentially using explicit waits.
6. **Evidence & Exit:** A PNG screenshot is saved to the artifacts directory, a single JSON string is printed to `stdout`, and the process terminates.
7. **Persistence:** Node.js parses the JSON string, saves a new `TestResult` record in MongoDB, and returns an HTTP 200 OK response with the saved record.

3. Test Execution Activity Diagram

The activity diagram specifies the workflow and operational control logic executed within the Python Selenium worker, highlighting decision branching, exception handling, and guaranteed cleanup.

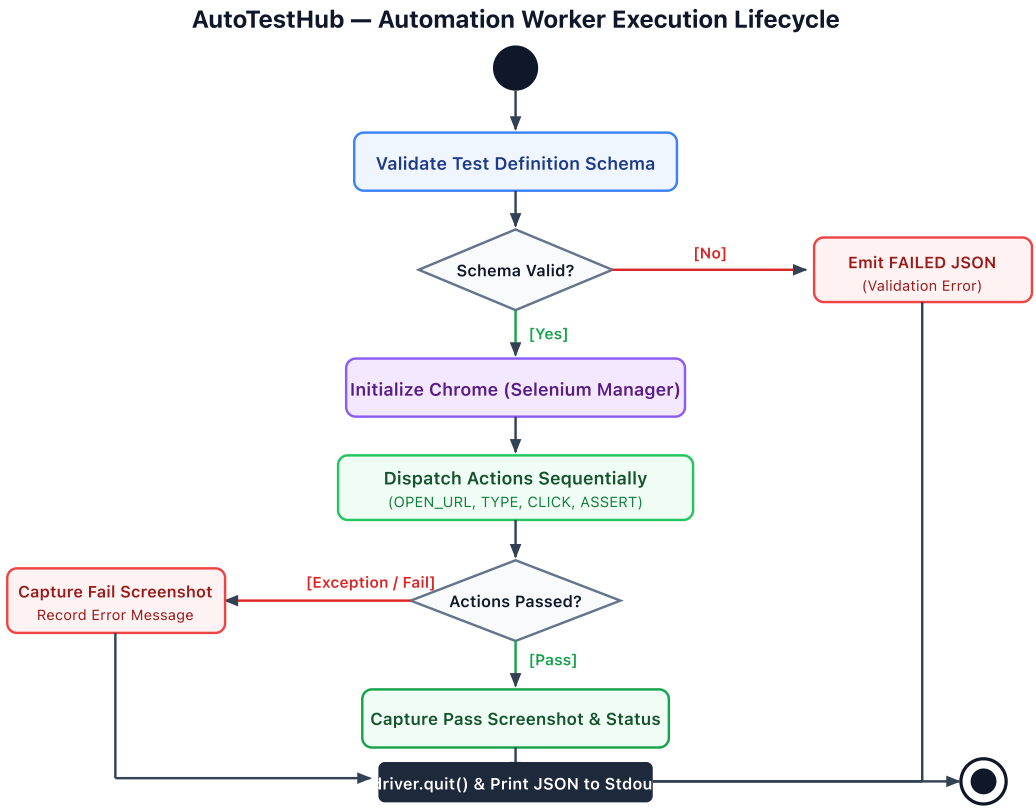


Figure 3: Activity Diagram detailing decision flows, error branches, screenshot capture, and cleanup.

3.1 Operational Control Flow Highlights

- **Fail-Fast Schema Validation:** Before browser launch, inputs are rigorously validated. If malformed or containing an unsafe URL scheme, execution terminates immediately with a structured error.
- **Action Dispatch Loop:** Supported actions (`OPEN_URL`, `TYPE`, `CLICK`, `ASSERT_TITLE`) are evaluated sequentially. Any timeout or assertion mismatch branches into the failure path.
- **Universal Screenshot Generation:** Screenshots are captured on both success and failure, ensuring visual auditability.
- **Guaranteed Process Cleanup:** `driver.quit()` is unconditionally invoked inside a `finally` block, ensuring zero resource leakage.